

Tugas 9: Praktikum Mandiri 9 Machine Learning

Hisyam Wildan Alfath - 0110222206

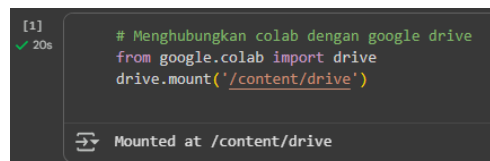
Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum mandiri ini bertujuan untuk membangun model klasifikasi kanker payudara menggunakan algoritma Gaussian Naive Bayes. Dataset dianalisis melalui tahap pembersihan data, eksplorasi, visualisasi, encoding label, standarisasi fitur, serta pembagian data latih dan uji. Hasil evaluasi menunjukkan bahwa model mencapai akurasi 92% pada data uji dan memiliki performa stabil berdasarkan cross validation dengan rata-rata akurasi sekitar 93%. Secara keseluruhan, model ini menunjukkan efektivitas yang baik dalam mengklasifikasikan jenis diagnosis kanker payudara.

1. Praktikum Mandiri 09

- Menghubungkan Colab dengan Google Drive

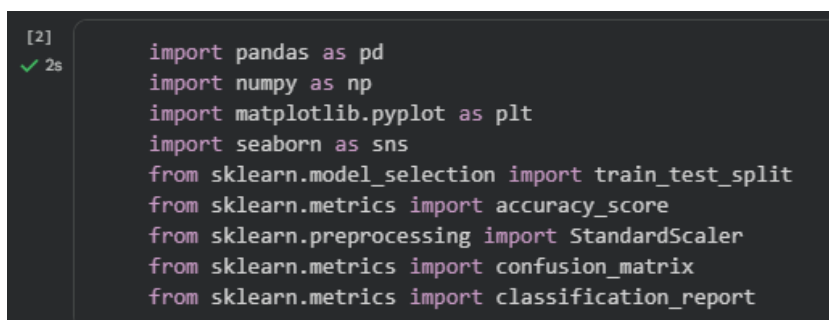


```
[1]
✓ 20s
# Menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Kode ini memanggil modul drive dari google.colab untuk menghubungkan (mount) Google Drive ke lingkungan Google Colab, dan outputnya adalah muncul pesan “Mounted at /content/drive” yang menandakan folder Google Drive sekarang sudah bisa diakses pada path tersebut di Colab.

- Mengimpor Library untuk Analisis Data dan Machine Learning



```
[2]
✓ 2s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Kode pada gambar digunakan untuk mengimpor berbagai library yang diperlukan dalam analisis data, visualisasi, dan pembuatan model machine learning,

seperti pandas dan numpy untuk pengolahan data, matplotlib dan seaborn untuk visualisasi, serta beberapa fungsi dari scikit-learn seperti `train_test_split`, `StandardScaler`, dan metrik evaluasi yang digunakan untuk membagi data, melakukan scaling, dan menilai performa model.

- Membaca Dataset CSV ke dalam DataFrame

```
[1]: titanic_data = pd.read_csv('/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 9/Praktikum dan Tugas Mandiri 09/data/data.csv')
```

Kode pada gambar digunakan untuk membaca file data berformat CSV menggunakan fungsi `pd.read_csv()`, di mana path file yang disimpan di Google Drive dimasukkan ke dalam fungsi tersebut, kemudian hasilnya disimpan dalam variabel `titanic_data` sebagai DataFrame yang siap digunakan untuk proses analisis lebih lanjut.

- Menampilkan Lima Baris Pertama Dataset

```
[4]: titanic_data.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33	184.60	2019.0
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	158.80	1956.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	152.50	1709.0
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	98.87	567.7
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	152.20	1575.0

5 rows x 33 columns

Kode `titanic_data.head()` digunakan untuk menampilkan lima baris pertama dari DataFrame sebagai langkah awal untuk melihat isi dan struktur data. Input dari perintah ini adalah DataFrame bernama `titanic_data` yang sebelumnya telah dibaca dari file CSV, sedangkan output-nya berupa tampilan 5 baris pertama lengkap dengan kolom seperti `id`, `diagnosis`, `radius_mean`, `texture_mean`, dan kolom lainnya, sehingga pengguna dapat melakukan pengecekan awal bahwa data berhasil dimuat dengan benar.

- Menampilkan Lima Baris Terakhir Dataset

```
[5]: titanic_data.tail()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...	26.40	166.10	2027.0
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...	38.25	155.00	1731.0
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...	34.12	126.70	1124.0
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...	39.42	184.60	1821.0
568	92751	B	7.76	24.54	47.52	181.0	0.05263	0.04362	0.00000	0.00000	...	30.37	59.16	268.6

5 rows x 33 columns

Kode `titanic_data.tail()` digunakan untuk melihat lima baris terakhir dari DataFrame sebagai pengecekan struktur dan isi data di bagian akhir. Input dari perintah ini adalah DataFrame `titanic_data` yang sudah dimuat sebelumnya, sedangkan output-nya berupa tampilan 5 baris terakhir beserta seluruh kolom seperti `id`, `diagnosis`,

radius_mean, hingga area_worst, sehingga memastikan bahwa data terbaca dengan lengkap sampai baris paling akhir.

- Menampilkan Informasi Struktur Dataset

```
[6]
✓ Os
titanic_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    569 non-null    int64
1   diagnosis                            569 non-null    object
2   radius_mean                          569 non-null    float64
3   texture_mean                         569 non-null    float64
4   perimeter_mean                      569 non-null    float64
5   area_mean                           569 non-null    float64
6   smoothness_mean                     569 non-null    float64
7   compactness_mean                    569 non-null    float64
8   concavity_mean                      569 non-null    float64
9   concave points_mean                 569 non-null    float64
10  symmetry_mean                       569 non-null    float64
11  fractal dimension_mean              569 non-null    float64
12  radius_se                           569 non-null    float64
13  texture_se                           569 non-null    float64
14  perimeter_se                        569 non-null    float64
15  area_se                             569 non-null    float64
16  smoothness_se                       569 non-null    float64
17  compactness_se                      569 non-null    float64
18  concavity_se                        569 non-null    float64
19  concave points_se                   569 non-null    float64
20  symmetry_se                         569 non-null    float64
21  fractal dimension_se                569 non-null    float64
22  radius_worst                        569 non-null    float64
23  texture_worst                       569 non-null    float64
24  perimeter_worst                     569 non-null    float64
25  area_worst                          569 non-null    float64
26  smoothness_worst                   569 non-null    float64
27  compactness_worst                   569 non-null    float64
28  concavity_worst                     569 non-null    float64
29  concave points_worst                569 non-null    float64
30  symmetry_worst                      569 non-null    float64
31  fractal dimension_worst              569 non-null    float64
32  Unnamed: 32                         0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Kode `titanic_data.info()` digunakan untuk melihat ringkasan struktur DataFrame, termasuk jumlah baris, jumlah kolom, tipe data tiap kolom, serta jumlah nilai non-null. Input dari perintah ini adalah DataFrame `titanic_data`, sedangkan output-nya berupa informasi lengkap seperti total 569 baris, 33 kolom, tipe data (float64, int64, object), dan penggunaan memori, sehingga membantu memahami kondisi data sebelum dilakukan analisis lebih lanjut.

- Mengecek Jumlah Nilai Kosong pada Dataset

A screenshot of a Jupyter Notebook cell showing the command `titanic_data.isnull().sum()` and its output. The output is a Series with 20 columns, all of which have a value of 0, indicating no missing data.

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0

Kode `titanic_data.isnull().sum()` digunakan untuk menghitung jumlah nilai kosong (missing values) pada setiap kolom dalam DataFrame. Input dari perintah ini adalah DataFrame `titanic_data`, sedangkan output-nya berupa daftar lengkap jumlah nilai kosong per kolom, yang pada hasil ini semuanya bernilai 0, sehingga menunjukkan bahwa dataset tidak memiliki data yang hilang dan siap untuk dianalisis lebih lanjut.

- Menghapus Kolom yang Tidak Diperlukan dari Dataset

A screenshot of two Jupyter Notebook cells. The first cell contains the code `titanic_data = titanic_data.drop(columns=['id'], axis=1)`. The second cell contains the code `titanic_data.isnull().sum()` followed by `titanic_data = titanic_data.drop(columns=['Unnamed: 32'], axis=1)`.

```
[8] titanic_data = titanic_data.drop(columns=['id'], axis=1)
[9] titanic_data.isnull().sum()
    titanic_data = titanic_data.drop(columns=['Unnamed: 32'], axis=1)
```

Kode pada gambar digunakan untuk menghapus kolom yang tidak relevan pada proses analisis. Pada baris pertama, input berupa DataFrame `titanic_data` dan kolom 'id' yang ingin dihapus, kemudian output-nya adalah DataFrame baru tanpa kolom tersebut. Pada bagian berikutnya, setelah mengecek kembali nilai kosong dengan `titanic_data.isnull().sum()`, kolom 'Unnamed: 32' juga dihapus karena tidak berisi data penting. Hasil akhirnya adalah DataFrame yang lebih bersih dan hanya berisi kolom-kolom yang diperlukan untuk analisis lanjutan.

- Menampilkan Statistik Deskriptif Dataset

```
[38]
✓ Os
titanic_data.describe()
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symetry_mean	fractal_dimension_mean	...
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.062798	...
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.007060	...
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960	...
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.057700	...
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.061540	...
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.066120	...
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.097440	...

8 rows x 30 columns

Kode `titanic_data.describe()` digunakan untuk menampilkan ringkasan statistik deskriptif dari seluruh kolom numerik dalam DataFrame. Input dari perintah ini adalah DataFrame `titanic_data`, sedangkan output-nya berupa tabel yang menunjukkan nilai seperti count, mean, std, min, 25%, 50%, 75%, dan max untuk masing-masing fitur. Hasil ini membantu memahami distribusi data, rentang nilai, serta pola awal sebelum dilakukan analisis atau pemodelan lebih lanjut.

- Melihat Distribusi Kelas Diagnosis

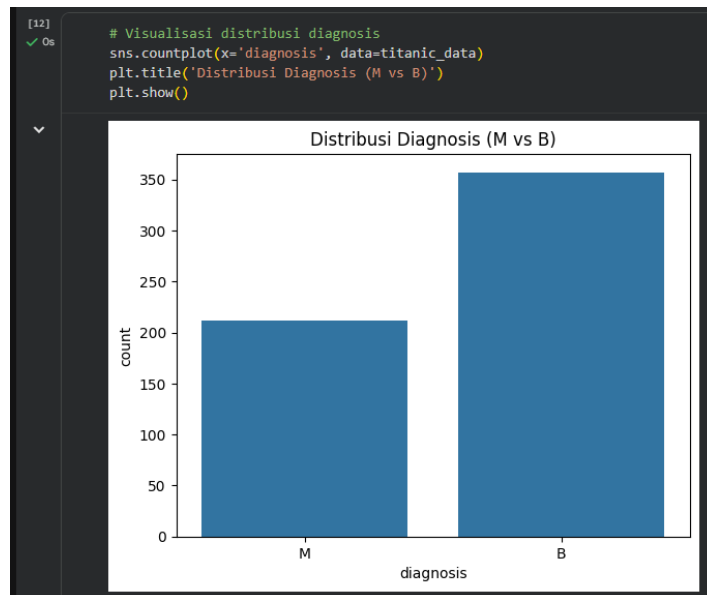
```
[11]
✓ Os
# Distribusi kelas diagnosis (M = Malignant, B = Benign)
titanic_data['diagnosis'].value_counts()
```

	count
diagnosis	
B	357
M	212

dtype: int64

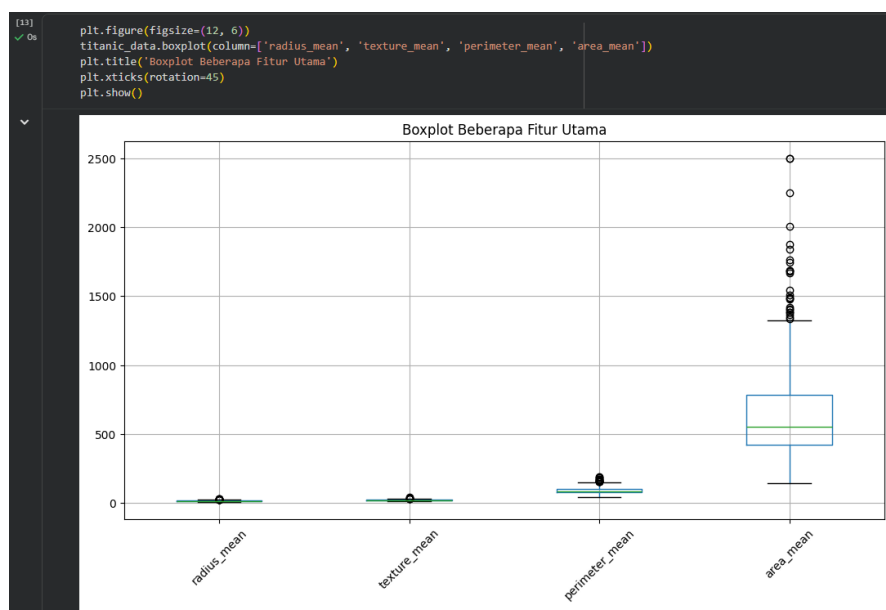
Kode `titanic_data['diagnosis'].value_counts()` digunakan untuk menghitung jumlah kemunculan setiap kategori pada kolom diagnosis. Input dari perintah ini adalah kolom diagnosis dalam DataFrame `titanic_data`, sedangkan output-nya berupa jumlah data untuk masing-masing kelas, yaitu B (Benign) sebanyak 357 dan M (Malignant) sebanyak 212, sehingga memberikan gambaran awal tentang keseimbangan kelas dalam dataset.

- Visualisasi Distribusi Kelas Diagnosis



Kode menggunakan `sns.countplot()` untuk menampilkan grafik batang yang menunjukkan jumlah masing-masing kelas pada kolom diagnosis. Input-nya adalah kolom 'diagnosis' dari DataFrame `titanic_data`, sedangkan output-nya berupa grafik distribusi yang memperlihatkan bahwa kelas B (Benign) lebih banyak dibandingkan M (Malignant). Visualisasi ini membantu memahami keseimbangan kelas sebelum melakukan proses pemodelan.

- Visualisasi Boxplot untuk Beberapa Fitur Utama



Kode pada gambar digunakan untuk menampilkan boxplot dari empat fitur utama, yaitu radius_mean, texture_mean, perimeter_mean, dan area_mean. Input-nya berupa DataFrame titanic_data beserta daftar kolom yang ingin divisualisasikan, sedangkan output-nya adalah grafik boxplot yang menunjukkan persebaran nilai, median, serta keberadaan outlier pada setiap fitur. Visualisasi ini membantu memahami variasi data dan mendeteksi nilai ekstrem yang mungkin mempengaruhi proses analisis atau pemodelan.

- Mengubah Label Diagnosis Menjadi Bentuk Numerik (Encoding)

```
[14]
✓ Os
replacements = {
    'diagnosis': {'M': 1, 'B': 0}
}

titanic_data.replace(replacements, inplace=True)

# Cek hasil encoding
titanic_data['diagnosis'].value_counts()

/tmp/ipython-input-1459475730.py:5: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version.
  titanic_data.replace(replacements, inplace=True)

count
diagnosis
0      357
1      212
dtype: int64
```

Kode pada gambar digunakan untuk melakukan label encoding pada kolom diagnosis, di mana nilai 'M' diganti menjadi 1 dan 'B' diganti menjadi 0. Input-nya adalah DataFrame titanic_data beserta dictionary replacements yang menentukan pasangan nilai lama dan baru. Output-nya berupa kolom diagnosis yang kini telah berubah menjadi format numerik, yang ditampilkan kembali dengan value_counts() dan menunjukkan bahwa jumlah kelas tetap sama yaitu 357 untuk 0 dan 212 untuk 1. Langkah ini penting agar data siap digunakan dalam algoritma machine learning yang membutuhkan input numerik.

- Memisahkan Fitur (X) dan Label (Y) untuk Proses Pemodelan

```
[15]
✓ Os
X = titanic_data.drop(columns=['diagnosis'])
Y = titanic_data['diagnosis']

X.head(), Y.head()
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	fractal_dimension_mean	...	radius_worst	texture_worst	perimeter_worst	\
0	0.07871	...	25.38	17.33	184.60	
1	0.05667	...	24.99	23.41	158.80	
2	0.05999	...	23.57	25.53	152.50	
3	0.09744	...	14.91	26.50	98.87	
4	0.05883	...	22.54	16.67	152.20	

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	concave points_worst	symmetry_worst	fractal_dimension_worst
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678


```
[5 rows x 30 columns],
0    1
1    1
2    1
3    1
4    1
Name: diagnosis, dtype: int64)
```

Kode pada gambar digunakan untuk memisahkan data menjadi dua bagian: X sebagai fitur dan Y sebagai label. Input-nya adalah DataFrame titanic_data, di mana kolom diagnosis dihapus untuk membentuk X, sedangkan kolom diagnosis sendiri disimpan sebagai Y. Output-nya berupa tampilan awal dari X yang berisi 30 fitur numerik, serta Y yang berisi nilai diagnosis (0 atau 1). Pemisahan ini merupakan langkah penting sebelum melakukan proses pelatihan model machine learning.

- Membagi Dataset Menjadi Data Latih dan Data Uji

```
[16] ✓ 0s X_train, X_test, Y_train, Y_test = train_test_split(  
    X, Y,  
    test_size=0.2,  
    stratify=Y,  
    random_state=42  
)  
  
print(X.shape, X_train.shape, X_test.shape)  
  
(569, 30) (455, 30) (114, 30)
```

Kode pada gambar menggunakan fungsi `train_test_split()` untuk membagi data menjadi `X_train`, `X_test`, `Y_train`, dan `Y_test`. Input-nya adalah fitur `X` dan label `Y`, dengan parameter `test_size=0.2` yang berarti 20% data digunakan untuk pengujian, serta `stratify=Y` agar proporsi kelas tetap seimbang. Output-nya terlihat pada hasil shape, yaitu `X` memiliki 569 sampel, `X_train` berisi 455 sampel, dan `X_test` berisi 114 sampel. Pembagian ini penting untuk memastikan model dapat diuji pada data yang tidak dilatih sebelumnya.

- Melakukan Standarisasi pada Data Latih dan Data Uji

```
[17] ✓ 0s scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Kode pada gambar menggunakan `StandardScaler` untuk menstandarisasi fitur agar memiliki rata-rata 0 dan standar deviasi 1. Input-nya adalah data latih (`X_train`) dan data uji (`X_test`). Output-nya berupa dua data baru: `X_train_scaled` yang diperoleh dari `fit_transform()`—menyesuaikan skala berdasarkan data latih—dan `X_test_scaled` yang distandarisasi menggunakan parameter yang sama melalui `transform()`. Proses ini penting agar model tidak dipengaruhi oleh perbedaan skala antar fitur.

- Melatih Model Gaussian Naive Bayes

```
[18] ✓ 0s from sklearn.naive_bayes import GaussianNB  
nb_model = GaussianNB()  
nb_model.fit(X_train_scaled, Y_train)
```

▼ GaussianNB ⓘ ?
GaussianNB()

Kode pada gambar digunakan untuk membuat dan melatih model klasifikasi Gaussian Naive Bayes. Input-nya adalah data latih yang sudah distandarisasi (X_train_scaled) serta label latih (Y_train). Output-nya berupa model nb_model yang telah terlatih setelah menjalankan fungsi .fit(). Model ini kemudian siap digunakan untuk melakukan prediksi pada data uji atau data baru.

- Mengukur Akurasi Model Gaussian Naive Bayes

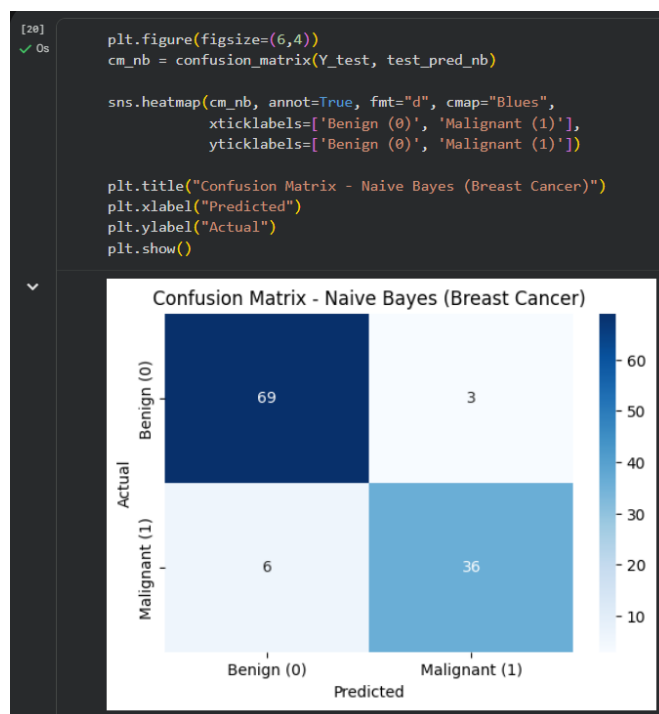
```
[19] ✓ Os
train_pred_nb = nb_model.predict(X_train_scaled)
test_pred_nb = nb_model.predict(X_test_scaled)

print("Training Accuracy (NB): ", accuracy_score(Y_train, train_pred_nb))
print("Testing Accuracy (NB): ", accuracy_score(Y_test, test_pred_nb))

▼ Training Accuracy (NB): 0.945054945054945
Testing Accuracy (NB): 0.9210526315789473
```

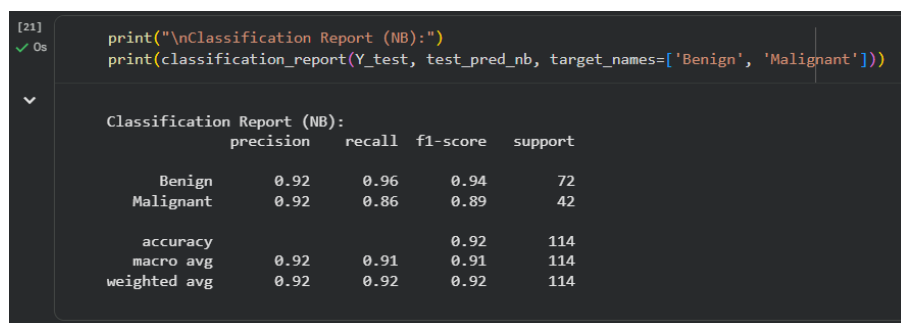
Kode pada gambar digunakan untuk melakukan prediksi menggunakan model Naive Bayes dan menghitung akurasi pada data latih dan data uji. Input-nya adalah model terlatih nb_model, serta data latih dan uji yang sudah distandarisasi (X_train_scaled dan X_test_scaled). Output-nya berupa nilai akurasi: 0.945 pada data latih dan 0.921 pada data uji, yang menunjukkan bahwa model memiliki performa prediksi yang cukup baik dan tidak mengalami overfitting yang signifikan.

- Menampilkan Confusion Matrix Model Gaussian Naive Bayes



Kode pada gambar digunakan untuk menghitung dan memvisualisasikan confusion matrix hasil prediksi model Naive Bayes. Input-nya adalah label aktual (Y_test) dan label prediksi (test_pred_nb). Output-nya berupa heatmap confusion matrix yang menunjukkan jumlah prediksi benar dan salah: 69 Benign benar, 36 Malignant benar, serta 3 salah prediksi Benign dan 6 salah prediksi Malignant. Visualisasi ini membantu memahami performa model secara lebih detail dibanding hanya melihat akurasi.

- Menampilkan Classification Report untuk Evaluasi Model



```
[21]
✓ 0s
print("\nClassification Report (NB):")
print(classification_report(Y_test, test_pred_nb, target_names=['Benign', 'Malignant']))
```

	precision	recall	f1-score	support
Benign	0.92	0.96	0.94	72
Malignant	0.92	0.86	0.89	42
accuracy			0.92	114
macro avg	0.92	0.91	0.91	114
weighted avg	0.92	0.92	0.92	114

Kode pada gambar digunakan untuk mencetak classification report dari hasil prediksi model Naive Bayes. Input-nya adalah label aktual (Y_test) dan label prediksi (test_pred_nb), beserta nama kelas yang ditampilkan. Output-nya berupa tabel evaluasi yang memuat nilai precision, recall, fl-score, dan support untuk kelas Benign dan Malignant. Hasil menunjukkan akurasi keseluruhan sebesar 0.92, dengan performa yang cukup seimbang antara kedua kelas, sehingga memberikan gambaran lengkap mengenai kualitas prediksi model.

- Evaluasi Model dengan Cross Validation (5-Fold)

```
[22] ✓ 0s from sklearn.model_selection import cross_val_score
cv_nb = cross_val_score(nb_model, X, Y, cv=5, scoring='accuracy')
print("\nNaive Bayes Cross Validation Accuracy (5-Fold):")
print("Scores:", cv_nb)
print("Mean Accuracy:", cv_nb.mean())
print("Std Deviation:", cv_nb.std())
```

Naive Bayes Cross Validation Accuracy (5-Fold):

Scores: [0.92105263 0.92105263 0.94736842 0.94736842 0.95575221]

Mean Accuracy: 0.9385188635305075

Std Deviation: 0.014585994424363306

Kode pada gambar digunakan untuk mengukur performa model Naive Bayes menggunakan teknik cross validation sebanyak 5 fold. Input-nya adalah model

nb_model, fitur X, label Y, serta parameter cv=5 dan scoring='accuracy'. Output-nya berupa lima skor akurasi untuk tiap fold, nilai mean accuracy sebesar 0.938, serta standard deviation sekitar 0.014. Hasil ini menunjukkan bahwa model stabil dan konsisten pada berbagai subset data, sehingga memperkuat keandalan performanya.

- Kesimpulan

Setelah melakukan praktikum mandiri ini, dapat disimpulkan bahwa model Gaussian Naive Bayes mampu melakukan klasifikasi kanker payudara dengan cukup baik. Data berhasil dibersihkan, diproses, dan distandarisasi sebelum digunakan untuk pelatihan model. Model menunjukkan performa tinggi dengan akurasi uji 92%, hasil confusion matrix yang seimbang, serta cross validation yang stabil dengan mean accuracy sekitar 93%. Secara keseluruhan, proses ini menunjukkan bahwa Naive Bayes adalah algoritma yang efektif dan sederhana untuk masalah klasifikasi medis seperti deteksi kanker payudara.

Link Praktikum 09:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%209/Praktikum%20dan%20Tugas%20Mandiri%2009/notebooks/praktikum09.ipynb

Tugas Mandiri 09:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%209/Praktikum%20dan%20Tugas%20Mandiri%2009/notebooks/latihan09.ipynb